



Simone Rizzo  • 2°
CEO @ Inferentia | AI Engineer

22.4.2026

link al video di
Simone Rizzo

+ Segui

OpenAI che rilascia un nuovo modello Open Source ed è pure utile per la privacy?!

Sì, è successo davvero... e stavolta hanno centrato il punto!

Il problema è sempre lo stesso: vogliamo usare i modelli più potenti (GPT, Claude, Gemini, GLM, MiniMAX, Kimi, ecc) ma abbiamo il terrore di mandare dati sensibili sul cloud. Nomi, email, numeri di telefono, indirizzi... un incubo per la compliance! 🤖

Beh, OpenAI ha appena "droppato" Privacy Filter, un modello progettato specificamente per rendere anonimi i tuoi prompt e i tuoi documenti PRIMA di darli in pasto all'AI.

In pratica, è il "buttafuori" della tua privacy che sta sulla porta di casa tua (ovvero sui tuoi server locali). 🏠🔒

Ecco perché secondo me è una bomba:

- 100% Locale: Gira sulla tua macchina. Non deve chiamare nessuna API esterna per pulire i dati.

- Leggerissimo: Funziona alla grande sia su GPU che su CPU. Non ti serve un supercomputer per farlo girare, un laptop moderno basta e avanza! 💻

- Zero Leak: Identifica e rimuove (o sostituisce) i dati sensibili (PII) trasformando un documento "pericoloso" in un testo anonimo pronto per il cloud.

- Gratis e Libero: È open-source. Lo scarichi, lo integri nella tua pipeline e fine dei giochi.

💡 Il mio consiglio: Se lavori in azienda o hai una startup dove la gestione dei dati è delicata, questo è il pezzo del puzzle che ti mancava. Puoi finalmente sbloccare l'uso dei modelli in cloud senza rischiare una multa epocale o un leak di dati sensibili.

Lo sto già testando per alcuni flussi di lavoro e devo dire che la precisione nel beccare le entità sensibili è davvero notevole. OpenAI stavolta ha fatto un bel regalo alla community!

Trovate tutto qui sulla repo ufficiale: 📄 <https://lnkd.in/eBj8d26y>

Privacy Filter predicts spans across eight categories:

- private_person
- private_address
- private_email
- private_phone
- private_url
- private_date
- account_number
- secret

OpenAI Privacy Filter is a bidirectional^{1/7} token-classification model for **personally identifiable information (PII) detection and masking in text**. It is intended for high-throughput data **sanitization** workflows where teams need a model that they can **run on-premises that is fast, context-aware, and**

link al mio documento
word che spiega come
installarlo



The screenshot shows the OpenAI Privacy Filter interface. At the top, the OpenAI logo is on the left, and a button labeled "Nuovo modello open source" is on the right. The main heading reads "Privacy Filter: i tuoi dati ~~esposti~~ protetti prima del prompt." Below this, there are two side-by-side boxes. The left box, titled "Testo originale", contains the text: "Cliente Mario Rossi, nato il 12/04/1984, email m.rossi@acme.it, IBAN IT60 X054...". The right box, titled "Testo protetto", contains the text: "Cliente [PERSON], nato il [DATE], email [EMAIL], IBAN [ACCOUNT]". A vertical bar between the two boxes is labeled "PRIVACY FILTER". At the bottom of the interface, there are three sections: "Apache 2.0 · Gratis", "CPU · GPU · Browser", and "128.000 token di context".

link al repo *github* di
OpenAI → sezione
«How to use»

Il **Privacy Filter di OpenAI** è uno strumento pensato per **intercettare, riconoscere e oscurare informazioni personali o potenzialmente sensibili** presenti in un testo prima che questo venga utilizzato in un processo di elaborazione automatica, ad esempio in una pipeline di intelligenza artificiale, in un sistema RAG, in un dataset di addestramento, in un log applicativo o in un prompt destinato a un modello linguistico. Il suo scopo principale è **ridurre il rischio** che dati identificativi, come nomi, indirizzi, email, date di nascita, numeri di telefono o altri riferimenti personali, vengano trasmessi o conservati inutilmente. Dal punto di vista interno, il filtro non si limita a una semplice ricerca meccanica di parole o pattern, ma analizza il testo nel suo contesto, individua le **porzioni che possono rappresentare dati personali e le sostituisce con etichette, mascheramenti o forme redatte**. In questo senso, il Privacy Filter si colloca tra gli strumenti di protezione preventiva del dato: non modifica la logica del documento né ne elimina necessariamente il significato generale, ma interviene sugli elementi che potrebbero consentire l'identificazione di una persona o la diffusione di informazioni riservate. Occorre tuttavia considerarlo come uno strumento di **mitigazione del rischio**, non come una garanzia assoluta di anonimizzazione, perché il riconoscimento automatico può sempre produrre omissioni o oscuramenti eccessivi, soprattutto quando il dato personale è implicito, ambiguo o dipendente dal contesto.

Prove mie → vedi file word *Guida all'utilizzo di privacy-filter di OpenAI*

Tabella riassuntiva

Test	Dispositivo	Righe	Tempo secondi	Tempo leggibile	Throughput righe/sec	Esito
test_1000.txt	CPU	1.000	913,995	15 min 14 s	1,09	OK
test_1000.txt	GPU	1.000	18,931	19 s	52,82	OK
test_10000.txt	GPU	10.000	135,994	2 min 16 s	73,53	OK
test_50000.txt	GPU	50.000	669,741	11 min 10 s	74,66	OK
test_50000.txt	CPU	50.000	45.068,726	12 h 31 min 9 s	1,11	OK

eseguita
solo su GPU

Confronto CPU/GPU

Righe	CPU	GPU	Accelerazione GPU
1.000	913,995 s	18,931 s	circa 48,3×
50.000	45.068,726 s	669,741 s	circa 67,3×

returncode = 0

significa che il comando è terminato senza errore.

Prove mie → vedi file word *Guida all'utilizzo di privacy-filter di OpenAI*

Commento tecnico

Il comportamento è coerente. La CPU elabora circa **1,1 righe al secondo**, sia nel test da 1.000 righe sia in quello da 50.000 righe. Questo indica una scalabilità abbastanza lineare, ma con throughput molto basso.

La GPU, invece, dopo il costo iniziale di caricamento e inizializzazione, si stabilizza intorno a **74 righe al secondo** sui file più grandi. Infatti:

```
GPU 10000 ≈ 73,5 righe/sec
```

```
GPU 50000 ≈ 74,7 righe/sec
```

Questo significa che la GPU lavora in modo molto più efficiente quando il volume cresce. Il test da 1.000 righe è penalizzato dall'overhead iniziale: caricamento Python, modello, PyTorch, CUDA, lettura file e scrittura output. Sui 10.000 e 50.000 record, invece, l'overhead viene assorbito e il throughput reale emerge meglio.

CPU: adatta solo a test piccoli o emergenza
GPU: necessaria per carichi reali

olsixhindoli.ai

Stai caricando documenti
sensibili su Claude/GPT.. !!

Ecco come evitarlo



Proteggi dati sensibili con AI

Scopri come l'IA può oscurare dati sensibili nei tuoi documenti. Proteggi la tua privacy e conformati alle normative. Leggi ora!
#Privacy #SicurezzaDati

Keywords: ["AI oscuramento dati","protezione dati sensibili","sicurezza documenti AI","privacy legale","GDPR conformità","strumenti AI privacy","oscuramento automatico","gestione dati sensibili","soluzioni cybersecurity","protezione informazioni"]

summary:

field	value
-----	-----
examples	5
tokens	145
windows	5
window_tokens	145
padded_window_tokens	145
elapsed_s	27.0687
tokens_per_s	5.3567
window_tokens_per_s	5.3567
padded_window_tokens_per_s	5.3567
eval_mode	typed
inference_tokens_per_second	3.8387
loss	0.4305
token_accuracy	0.9448

detection_metrics:

metric	value
-----	-----
detection.precision	0.9697
detection.recall	1.0000
detection.f1	0.9846
detection.f2	0.9938
detection.span.precision	0.9375
detection.span.recall	0.8571
detection.span.f1	0.8955
detection.span.f2	0.8721

timings:

stage	seconds
-----	-----
startup	3.5524s
window_stage_wall	23.3929s
input_pipeline	0.0000s
model_forward	20.7888s
score_stitch	16.9076s
metrics_prepare	0.0439s
decode	0.0764s
prediction_assembly	0.0001s
metrics_accuracy	0.0001s
metrics_finalize	0.0001s
metrics_total	0.1207s
prediction_write	0.0000s
eval_core_total	23.5137s
eval_total_with_write	23.5137s
overall_wall	27.0764s

throughput:

metric	tokens_per_second
-----	-----
inference_only_tokens_per_second	3.8387
model_forward_tokens_per_second	6.9749
eval_total_tokens_per_second	6.1666

throughput_note: inference_only_tokens_per_second uses model_forward + score_stitch + decode + prediction_assembly.

Esempio OpenAI – 1/2

Cosa sta facendo questo comando

👉 Sta:

- prendendo 5 esempi annotati (ground truth)
- facendo girare il modello
- confrontando **predizioni vs verità**
- producendo **metriche di qualità + performance**

Sezione `summary`Esempio OpenAI –
2/2

examples: 5
tokens: 145

- 5 frasi analizzate
- 145 token totali

tokens_per_s: 5.3567
inference_tokens_per_second: 3.8387

👉 velocità:

- ~5 token/sec complessivi
- ~3.8 token/sec solo inferenza

⚠️ Molto lento → normale:

- GPU piccola (GT 1030)
- oppure CPU fallback parziale

loss: 0.4305
token_accuracy: 0.9448

- **loss** → errore medio del modello (più basso = meglio)
- **accuracy** ~94.5% → buono

Sezione `detection_metrics` (la più importante)

Questa è la qualità reale del sistema.

Precision

detection.precision: 0.9697

👉 quando il modello segnala un dato sensibile:

- ha ragione nel 97% dei casi

✓ pochi falsi positivi

Recall

detection.recall: 1.0000

👉 trova **tutti** i dati sensibili

✓ zero falsi negativi

F1 score

detection.f1: 0.9846

👉 sintesi tra precision e recall

✓ valore molto alto → modello molto buono

F2 score

detection.f2: 0.9938

👉 dà più peso al recall

✓ coerente con recall = 1.0

Span metrics (più severe)

Queste valutano se il modello trova **esattamente** il pezzo giusto di testo

detection.span.precision: 0.9375
detection.span.recall: 0.8571
detection.span.f1: 0.8955

👉 più basse → normale

Perché:

- non basta trovare "una data"
- deve trovare **esattamente** i caratteri giusti